Especificação SDD — TaskAnalyzer

Versão: 1.2 (Fase 3: CI/CD, Docker e governança) · **Versões anteriores:** 1.1 (Fase 2), 1.0 (Fase 1)

> Esta especificação é a **fonte da verdade** para a geração de código. O que não está neste contrato não deve ser implementado.

---

0. Histórico de versões

Versão	Data	Mudança
1.0	12/09/2026	Especificação consolidada da Fase 1.
1.1	17/10/2026	Contrato alinhado ao enunciado da Fase 2 (ver tabela abaixo). A spec foi revisada **antes** de qualquer geração de código.
1.2	24/10/2026	Fase 3: fluxo com `develop`, pipeline de CI, execução em container e artefatos de rastreabilidade. O contrato do módulo (seções 1 a 4) não muda.

Mudanças da v1.1 para a v1.2:

Item	v1.1	v1.2	Motivo
Fluxo Git	`main` + `feature/*`	`feature/*` → `develop` → `main`	O enunciado da Fase 3 dispara o CI em `main` e `develop`.
Integração contínua	Testes só locais	GitHub Actions a cada push e PR (seção 5.2)	Pilar 1 da Fase 3.
Execução isolada	`.venv` local	`Dockerfile` + `docker-compose.yml` (seção 5.3)	Pilar 2 da Fase 3.
Artefatos de spec	Só este `.md`	Mais `requirements.yaml` e `system_design_document.pdf` (seção 5.5)	Árvore de diretórios exigida pela Fase 3.
Critério de PR	Testes locais verdes	Também o check verde do GitHub Actions	Homologação passa a ter prova automatizada e pública.

Mudanças da v1.0 para a v1.1:

Item	v1.0	v1.1	Motivo
Função pública	`analisar_tarefas`	`analyze_tasks`	Nome exigido pelo enunciado da Fase 2.
Exceções	`ListaDeTarefasVaziaError`, `TarefaInvalidaError`, `AusenciaDeTarefasConcluidasError`	`TaskValidationError` (herda de `TaskAnalyzerError`)	O enunciado exige `TaskValidationError` e o retorno `0.0` para entradas vazias.
Unidade de tempo	minutos (`tempo_medio_conclusao_min`)	horas (`tempo_medio_conclusao_horas`)	Campo exigido pelo enunciado.

Item	v1.0	v1.1	Motivo
Contadores	`quantidade_tarefas_recebidas`, `quantidade_tarefas_consideradas`	`total_tarefas`, `total_concluidas`, `total_pendentes`	Campos exigidos pelo enunciado.
Lista vazia / sem concluídas	lança exceção	retorna relatório com métricas `0.0`	O enunciado exige o retorno `0.0` sem exceção. A divisão por zero continua impedida (RN-08).
Prioridade sem tarefas	`tempo` e `atraso` = `None`	`0.0`	Mesma regra da linha anterior, para manter consistência.
Cronologia	conclusão < início é inválida	também conclusão < **criação** (Cenário 2 do enunciado)	Cenário de erro exigido pelo enunciado.
Git Flow	`main`, `develop`, `feature/*`	`main` + `feature/*`	Estratégia exigida pelo enunciado.

---

1. Visão geral e contrato de negócio

1.1 Propósito

O **TaskAnalyzer** é um motor de análise de produtividade que troca o "achismo" por dados auditáveis. Ele recebe uma lista de tarefas processadas por um sistema externo e devolve um relatório com totais, tempo médio de conclusão e taxa de atraso, gerais e por prioridade.

1.2 Requisitos

ID	Requisito	Tipo
REQ-01	Analisar listas de tarefas e produzir métricas consolidadas.	Funcional
REQ-02	Calcular o tempo médio de conclusão (em horas) das tarefas concluídas.	Funcional
REQ-03	Calcular a taxa de tarefas concluídas após o prazo.	Funcional
REQ-04	Segmentar quantidade, tempo médio e taxa de atraso por prioridade.	Funcional
REQ-05	Operar exclusivamente em memória, sem efeitos colaterais.	Não funcional
REQ-06	Determinismo: a mesma entrada gera exatamente a mesma saída.	Não funcional

1.3 Fora de escopo

Tudo que a especificação silencia, a IA tende a interpretar como liberdade criativa. Por isso, o módulo **NÃO** realiza:

1. Criação, edição, exclusão ou persistência de tarefas (banco de dados ou disco).
2. Exposição de APIs HTTP, GUI ou CLI.
3. Autenticação, autorização ou integração com serviços externos/redes.

1.4 Vocabulário de domínio

Termo	Definição operacional
Tarefa	Unidade de trabalho com identificador único, título, prioridade, status e marcos temporais.

Termo	Definição operacional
Tarefa concluída	Tarefa com `status == "concluida"` e `data_inicio` e `data_conclusao` preenchidos.
Tarefa pendente	Tarefa com `status == "pendente"`.
Tempo de conclusão	Intervalo exato, em **horas**, entre `data_inicio` e `data_conclusao`.
Tarefa em atraso	Tarefa concluída cuja `data_conclusao` é **estritamente posterior** ao `prazo`.
Taxa de atraso	Percentual de tarefas em atraso sobre o total de tarefas concluídas.

---

2. Contrato executável de interface

2.1 Entrada

Uma lista (`list[Tarefa]`). `Tarefa` é `@dataclass(frozen=True)` com os campos abaixo. A construção deve usar argumentos nomeados.

Campo	Tipo	Obrigatoriedade	Restrições de validação
`id_tarefa`	`int`	Obrigatório	Maior que zero e único na lista. `bool` não é aceito.
`titulo`	`str`	Obrigatório	Não vazio, não só espaços, no máximo 200 caracteres.
`prioridade`	`str`	Obrigatório	Domínio: `baixa`, `media`, `alta` (minúsculas, sem acento).
`status`	`str`	Obrigatório	Domínio: `pendente`, `concluida`, `cancelada`.
`data_criacao`	`datetime`	Obrigatório	*Timezone-aware* em UTC.
`prazo`	`datetime`	Obrigatório	*Timezone-aware* em UTC.
`data_inicio`	`datetime | None`	Condicional	Exigida se `status == "concluida"`. Não pode ser anterior a `data_criacao`.
`data_conclusao`	`datetime | None`	Condicional	Exigida se `status == "concluida"`. Não pode ser anterior a `data_criacao` nem a `data_inicio`.

Nota de engenharia: datas *naive* ou fora de UTC são rejeitadas. Isso elimina falhas silenciosas de fuso horário e garante testes determinísticos.

2.2 Saída

`RelatorioProdutividade` (`@dataclass(frozen=True)`):

Campo	Tipo	Descrição
`total_tarefas`	`int`	Total de elementos na lista de entrada.
`total_concluidas`	`int`	Tarefas com status `concluida`.
`total_pendentes`	`int`	Tarefas com status `pendente`.
`tempo_medio_conclusao_horas`	`float`	Média geral, em horas, 2 casas decimais.
`taxa_atraso_percentual`	`float`	Percentual geral (0.0 a 100.0), 2 casas decimais.
`indicadores_por_prioridade`	`dict[str, IndicadorPrioridade]`	Chaves fixas: `baixa`, `media`, `alta`.

`IndicadorPrioridade` (`@dataclass(frozen=True)`): `quantidade: int` (concluídas daquela prioridade), `tempo_medio_conclusao_horas: float`, `taxa_atraso_percentual: float`.

2.3 Assinatura pública

```
def analyze_tasks(tarefas: list[Tarefa]) -> RelatorioProdutividade:
    """Analisa uma lista de tarefas e retorna métricas de produtividade.

    Args:
        tarefas: Lista de tarefas a serem analisadas. Pode ser vazia.

    Returns:
        Relatório com totais, métricas gerais e indicadores por prioridade.

    Raises:
        TaskValidationError: Se alguma tarefa violar as restrições do contrato.
    """
```

2.4 Regras de negócio

- **RN-01:** Só tarefas `concluida` entram no cálculo de tempo e atraso. As demais só entram nos contadores `total_tarefas` e `total_pendentes`.
- **RN-02:** Tempo de conclusão = `data_conclusao − data_inicio`, convertido para horas.
- **RN-03:** Há atraso se `data_conclusao > prazo`. Concluir no instante exato do prazo **não** é atraso.
- **RN-04:** O tempo médio é a média aritmética simples dos tempos das tarefas concluídas.
- **RN-05:** Taxa de atraso = `(tarefas_em_atraso / total_concluidas) * 100`.
- **RN-06:** Todo `float` da saída é arredondado para 2 casas decimais, só no final (as médias usam os valores sem arredondamento).
- **RN-07:** Os indicadores por prioridade aplicam as RN-02 a RN-06 ao subconjunto da prioridade.
- **RN-08:** Sem tarefas concluídas (no geral ou numa prioridade), as médias e taxas valem `0.0`. Nenhuma divisão por zero pode ocorrer.
- **RN-09:** O dicionário de prioridades sempre tem as 3 chaves.
- **RN-10:** Validação *fail-fast*: uma única tarefa inválida aborta a análise com `TaskValidationError`.
- **RN-11:** A ordem da lista de entrada não altera o resultado.
- **RN-12:** A execução é pura: não altera a lista original nem seus objetos.
- **RN-13:** Em tarefas `pendente` ou `cancelada`, `data_inicio` e `data_conclusao` são opcionais. Se informadas, seguem as mesmas regras de UTC e cronologia.

2.5 Tratamento de exceções

Proibido lançar `Exception` ou `ValueError` genéricos. Hierarquia:

- `TaskAnalyzerError(Exception)`: base do módulo.
- `TaskValidationError(TaskAnalyzerError)`: campo ausente ou de tipo errado, valor fora do domínio, id duplicado, data *naive*/fora de UTC ou anomalia cronológica. Expõe os atributos `id_tarefa` (ou `None` se não for possível identificar a tarefa) e `motivo`, e a mensagem cita os dois.

---

3. Cenários de aceite (BDD)

Cenário 1 — Sucesso: cálculo geral e segmentado

Dado uma lista de 5 tarefas, todas criadas em 2026-01-10 às 08:00 UTC:

id	prioridade	status	início	conclusão	prazo	Resultado
1	alta	concluida	10/01 09:00	10/01 11:00	10/01 18:00	2h, no prazo
2	alta	concluida	10/01 08:00	10/01 12:00	10/01 10:00	4h, atrasada
3	media	concluida	10/01 14:00	10/01 14:30	11/01 12:00	0,5h, no prazo
4	baixa	pendente	—	—	12/01 18:00	—
5	baixa	cancelada	—	—	12/01 18:00	—

Quando `analyze_tasks()` for executada, **Então** o relatório contém exatamente:

- `total_tarefas = 5`, `total_concluidas = 3`, `total_pendentes = 1`
- `tempo_medio_conclusao_horas = 2.17` (média de 2, 4 e 0,5 = 2,1666…)
- `taxa_atraso_percentual = 33.33` (1 atraso em 3)
- `alta`: quantidade 2, tempo 3.0, atraso 50.0
- `media`: quantidade 1, tempo 0.5, atraso 0.0
- `baixa`: quantidade 0, tempo 0.0, atraso 0.0

Cenário 2 — Erro: datas inconsistentes

Dado uma tarefa concluída cuja `data_conclusao` é anterior à `data_criacao`, **Quando** `analyze_tasks()` for executada, **Então** dispara `TaskValidationError` com o `id_tarefa` e o motivo.

Casos de borda e de validação

ID	Dado	Então
CB-01	Lista vazia	Relatório com totais `0` e métricas `0.0`, sem exceção.
CB-02	Apenas tarefas pendentes	Totais corretos, métricas `0.0`, sem exceção.
CB-03	Conclusão no instante exato do prazo	Não é atraso (`0.0`).
CB-04	Conclusão anterior ao início	`TaskValidationError`.
CB-05	Prioridade ou status fora do domínio (ex.: `"urgente"`)	`TaskValidationError`.
CB-06	Dois ids iguais	`TaskValidationError`.
CB-07	Data *naive* ou fora de UTC	`TaskValidationError`.
CB-08	Tarefa concluída sem `data_inicio` / `data_conclusao`	`TaskValidationError`.

ID	Dado	Então
CB-09	Título vazio, só espaços ou com mais de 200 caracteres	`TaskValidationError`.
CB-10	A mesma lista em outra ordem	Relatório idêntico (RN-11).
CB-11	Lista de entrada após a análise	Igual a uma `deepcopy` feita antes (RN-12).

---

4. Test Harness (pytest)

Os testes ficam em `tests/test_harness.py` e são escritos **antes** da implementação (TDD).

Cenário	Técnica
Cenário 1	Fixture com as 5 tarefas, asserção de cada campo do relatório.
Cenário 2	`pytest.raises(TaskValidationError)` e verificação de `id_tarefa`/`motivo`.
CB-01, CB-02	Asserção de totais e métricas `0.0`.
CB-03	Igualdade de timestamps entre conclusão e prazo.
CB-04 a CB-09	`@pytest.mark.parametrize` com tarefas corrompidas.
CB-10, CB-11	Lista invertida e `copy.deepcopy`.

Meta obrigatória: cobertura de **90% ou mais** (`pytest-cov`), sem depender do relógio do sistema (`datetime.now()` é proibido).

---

5. Repositório, infraestrutura e homologação

A estrutura do repositório está em `README.md` e as regras para a IA, em `CONTEXT_RULES.md`.

5.1 Fluxo Git

- `feature/*`: uma branch por entrega, sempre criada a partir da `develop`.
- `develop`: integração. Recebe as features por Pull Request.
- `main`: versão homologada. Recebe apenas Pull Requests vindos da `develop`.

5.2 Integração contínua (GitHub Actions)

O arquivo `.github/workflows/ci.yml` é o contrato do pipeline:

Etapa	Definição
Gatilhos	`push` e `pull_request` nas branches `main` e `develop`
Ambiente	`ubuntu-latest` com Python 3.11
Preparação	`pip install -r requirements.txt`
Execução	`pytest tests/test_harness.py -v`

O pipeline usa o mesmo `pytest.ini` do ambiente local. Por isso, ele também falha se a cobertura ficar abaixo de 90%.

5.3 Execução isolada (Docker)

- `Dockerfile`: imagem base `python:3.11-slim`. Instala o `requirements.txt`, copia `src/`, `tests/` e `pytest.ini`, roda como usuário não-root e tem `CMD ["pytest", "tests/test_harness.py", "-v"]`.
- `docker-compose.yml`: define o serviço `tests`. A suíte inteira roda com um único comando: `docker compose run --rm tests`.
- O resultado no container deve ser idêntico ao local: todos os testes aprovados e cobertura de pelo menos 90%.

5.4 Critérios de aceite de Pull Request

1. Test Harness 100% verde no ambiente local.
2. Check verde do GitHub Actions no PR.
3. Cobertura de pelo menos 90%.
4. O revisor humano confirma que nenhuma regra das CONTEXT_RULES foi violada.
5. O PR descreve *o que foi gerado pela IA* e *o que precisou de intervenção humana*.

5.5 Artefatos de especificação

Arquivo	Papel
`specs/task_analyzer_spec.md`	Fonte da verdade, editada por PR.
`specs/requirements.yaml`	Rastreabilidade: cada requisito, regra e cenário ligado ao teste que o cobre. É documentação; nenhum código o lê, pois isso exigiria uma biblioteca de terceiros (PR-01).
`specs/system_design_document.pdf`	Este documento em PDF, gerado a partir do `.md`. Deve ser gerado de novo sempre que o `.md` mudar.

Código que só passa nos testes não está homologado. Homologar exige que o arquiteto humano entenda cada linha produzida pela IA e se responsabilize por ela.